

Clean India

Civic Issue Reporting Platform

CI/CD Automation

Kubernetes Orchestration

Real-Time Monitoring

Submitted By

Prabhav Stambamkadi

DevOps Project · B.Tech CSE



Introduction to Clean India

<> What is Clean India?

Clean India is a cloud-native MERN stack web platform designed to demonstrate modern DevOps practices — from development to containerized cloud deployment with full observability.

Motivation

Traditional deployment processes are manual, slow, error-prone and difficult to monitor. There is no visibility into build failures or production health.



Objectives

- Automate software delivery
- Containerize with Docker
- Deploy using Kubernetes
- Monitor health & metrics



Expected Outcomes

- Faster deployment cycles
- Higher reliability
- Better scalability
- Complete observability

Technologies Used

Infrastructure



AWS EC2



Terraform

Version Control



Git



GitHub

CI/CD



Jenkins

Containerization



Docker



Docker Hub

Orchestration



Kubernetes



kubect!

Monitoring



Prometheus



Grafana

Application Stack



ReactJS



NodeJS



ExpressJS



MongoDB

System Architecture

 **Developer**
Code Commit

Developer writes and commits code to the local environment

 **GitHub Repository**
Source Control

Code pushed to GitHub — triggers webhook to Jenkins automatically

 **Jenkins CI/CD**
Build & Test

Jenkins pulls code, runs automated builds and tests via Jenkinsfile

 **Docker Build**
Image Creation

Docker builds containerized images for frontend and backend

 **Docker Hub Registry**
Image Storage

Built images pushed to Docker Hub for version-controlled storage

 **Kubernetes Cluster**
Orchestration

Kubernetes pulls images and deploys pods with ReplicaSets & Services

 **Clean India App**
Production

Application runs in production — NodePort exposes it externally

 **Prometheus**
Metrics Collection

Prometheus scrapes metrics from all containers and services

 **Grafana Dashboard**
Visualization

Grafana connects to Prometheus and displays real-time dashboards



Source Code Management with GitHub



Repository Hosting

Complete project source — frontend, backend, kubernetes, monitoring, terraform folders



Branch Management

master branch with structured commits and version tagging



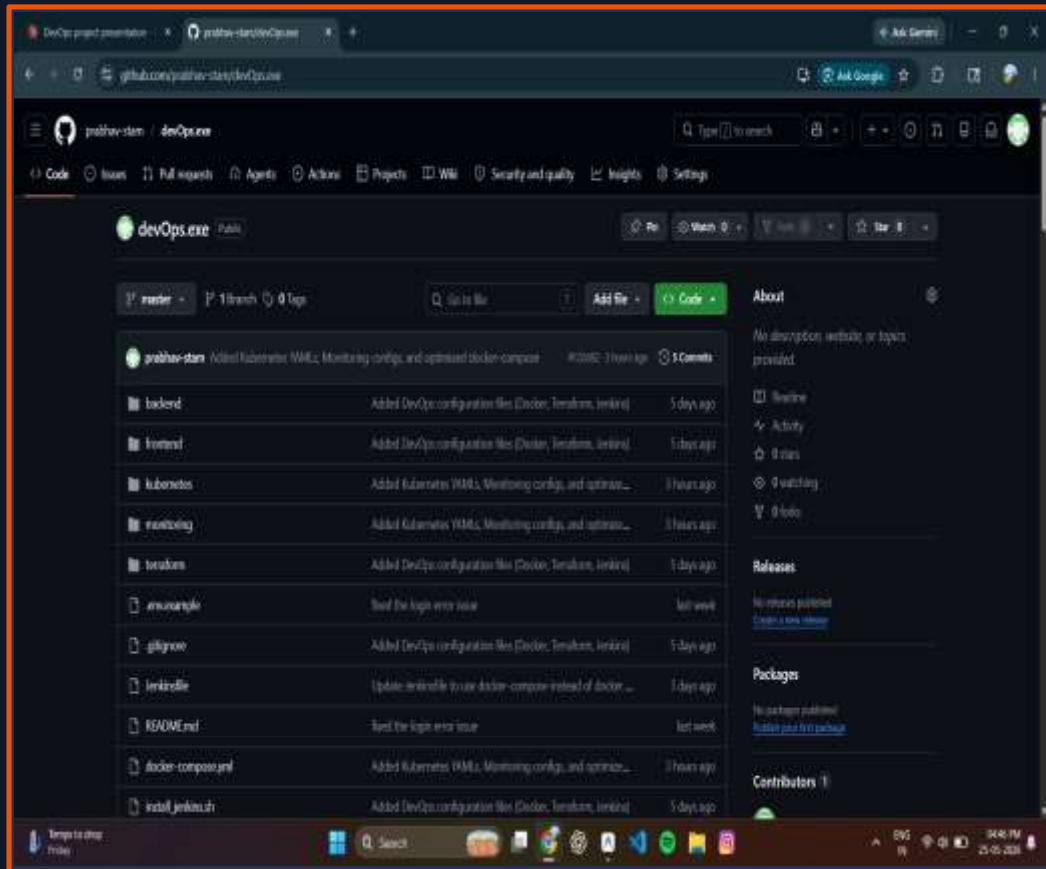
Webhook Integration

GitHub webhook triggers Jenkins CI/CD pipeline on every push



Version Control

5 commits tracking complete project evolution from code to DevOps config





Continuous Integration & Deployment



Code Commit



Webhook Trigger



Jenkins Build



Docker Build



Push to Hub



K8s Deploy

Automation

Zero manual build steps

Faster Releases

Build to deploy in ~3min 28sec

Consistency

Same pipeline every run

Visibility

Build #8 logged & tracked

The screenshot shows the Jenkins web interface. On the left is a sidebar with navigation links: 'New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. Below these are sections for 'Build Queue' (showing 'No builds in the queue') and 'Build Executor Status' (showing '0/2' executors). The main area displays a table of build history for a project named 'cleanindia-pipeline'. The table has columns for status (S for Success, W for Waiting), name, last success time, last failure time, and last duration. Build #8 is highlighted in blue, indicating it is the current build. A search bar and 'Add description' button are at the top right.

S	W	Name	Last Success	Last Failure	Last Duration
✓	⚡	cleanindia-pipeline	3 hr 21 min	3 hr 32 min	3 min 28 sec



Docker-Based Application Packaging

Containers



Frontend Container

React + Nginx (Port 80)



Backend Container

Node.js + Express (3000)



MongoDB Container

Database layer



Prometheus Container

Metrics scraper



Grafana Container

Dashboard UI

Dockerfile Highlights

Base Image: node:20-alpine (backend) · nginx:alpine (frontend)

Multi-stage: Separate build and runtime layers for minimal image size

Dependencies: npm install runs in build stage only

Env Vars: Injected at runtime via docker-compose.yml

Port: 80 (frontend), 3000 (backend), 27017 (MongoDB)



Portability



Isolation



Lightweight



Consistency



Container Orchestration using Kubernetes

Kubernetes Cluster (Minikube)

Namespace: default

Deployment

Manages pod lifecycle

ReplicaSet

Ensures desired replicas

Pod × 2

Frontend + Backend

Service

NodePort exposure

ConfigMap

Environment config

NodePort

Port 30000+ external



Self-Healing

Pods automatically restart on failure — zero downtime



Auto-restart

Kubernetes controller ensures desired state is maintained



Scaling

Scale pods horizontally with kubectl scale commands



Load Balancing

Service distributes traffic evenly across all pods



Declarative YAML

All resources defined as code in kubernetes/ folder



Infrastructure Monitoring with Prometheus

Metrics Collection

Scrapes metrics from Node Exporter, Docker containers and app endpoints

Scrape Targets

Configured via prometheus.yml — currently 1/1 targets UP at localhost:9090

Time-Series Database

Stores metrics with timestamp for historical analysis and trend detection

Alert Generation

Threshold-based alerts can trigger notifications via Alertmanager

Live Target Status

Endpoint: http://localhost:9090/metrics

Status: 1 / 1 UP ✓

Last Scrape: 11.347 seconds ago

Duration: 11ms

Job: prometheus

CPU Usage

Memory Usage

Disk I/O

Network

Container

Prometheus

Query Alerts Status > Target health

Select scrape pool Filter by target health Filter by endpoint or labels

Endpoint	Labels	Last scrape	State
http://localhost:9090/metrics	instance="localhost:9090" job="prometheus"	11.347s ago 11ms	1 / 1 up



Real-Time Visualization with Grafana

Prometheus Version

3.11.3

Instance Down

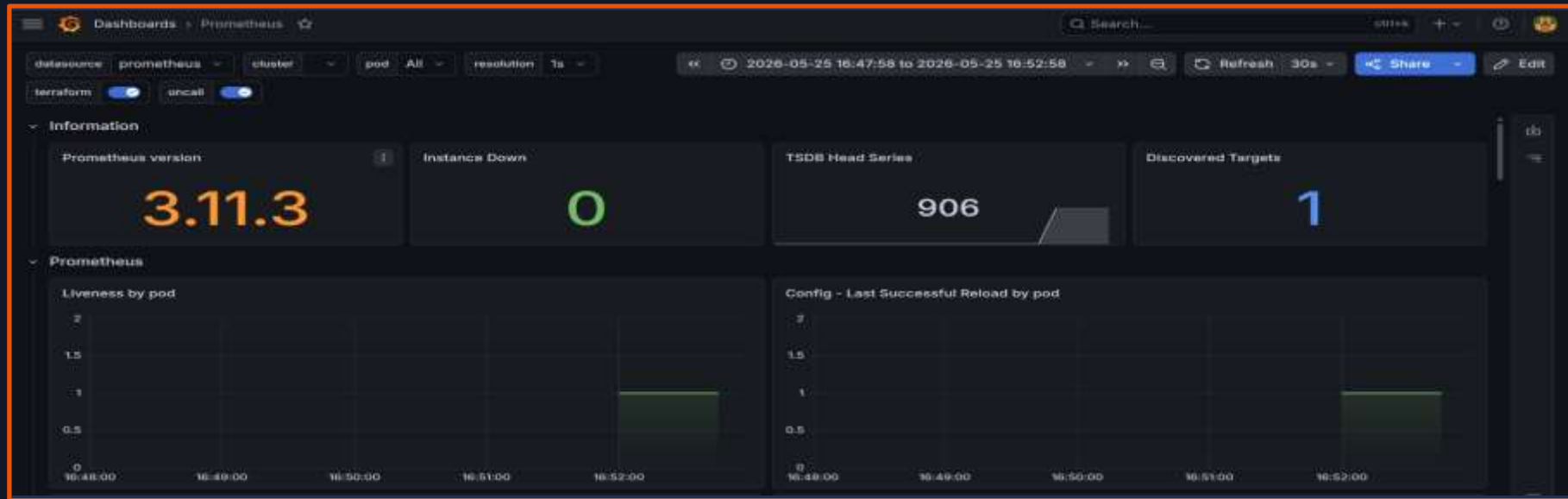
0

TSDb Head Series

906

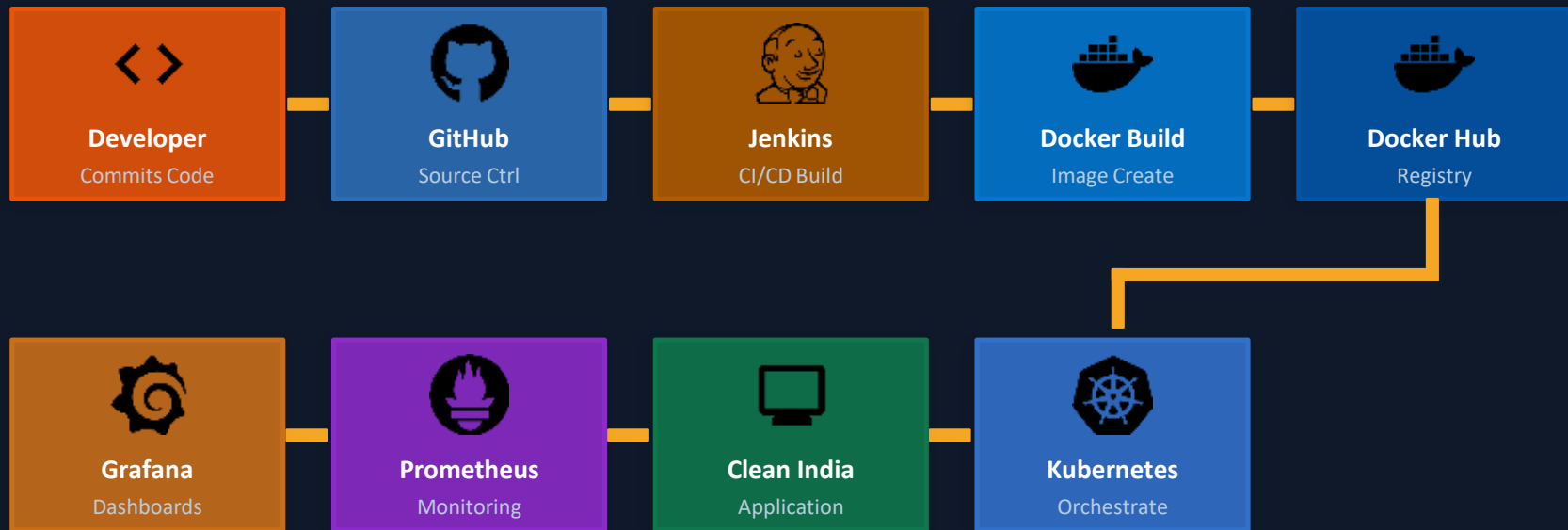
Discovered Targets

1



Metrics: node_cpu_seconds_total · node_memory_MemAvailable_bytes · Liveness by pod · Config Last Successful Reload by pod

Complete DevOps Workflow



This workflow automates the entire software delivery lifecycle from code commit to deployment and monitoring — ensuring reliability, scalability and operational visibility.

Benefits & Outcomes



Automated Deployment

CI/CD pipeline delivers from commit to production in ~3.5 minutes



Faster Software Delivery

Webhooks, automated builds and zero-touch deployments



Scalable Infrastructure

Kubernetes scales pods horizontally based on demand



Containerized Env

Docker ensures identical runtime across dev, test and production



Centralized Monitoring

Prometheus + Grafana provide unified visibility across all services



Improved Reliability

Kubernetes self-healing and ReplicaSets prevent downtime



Reduced Downtime

Zero-downtime rolling updates via Kubernetes deployment strategy

Future Enhancements

01



Auto Scaling

HPA — auto-scale pods based on CPU/memory load

02



GitOps with ArgoCD

Declarative CD with Git as single source of truth

03



Kubernetes Ingress

NGINX Ingress Controller for domain routing + SSL

04



Monitoring Alerts

Alertmanager integration for incident notification

05



Multi-Node Cluster

Production-grade K8s cluster on multiple EC2 nodes

06



Cloud-Native Security

RBAC, Secrets management, OPA policy enforcement

07



CI/CD Optimization

GitHub Actions parallel builds, caching strategies

08



Service Mesh (Istio)

mTLS, traffic management, distributed tracing

Thank You!

Clean India: Civic Issue Reporting Platform

Docker

Kubernetes

Jenkins

Prometheus

Grafana

Terraform

Prabhav Stambamkadi

Lovely Professional University · B.Tech CSE · DevOps Project

